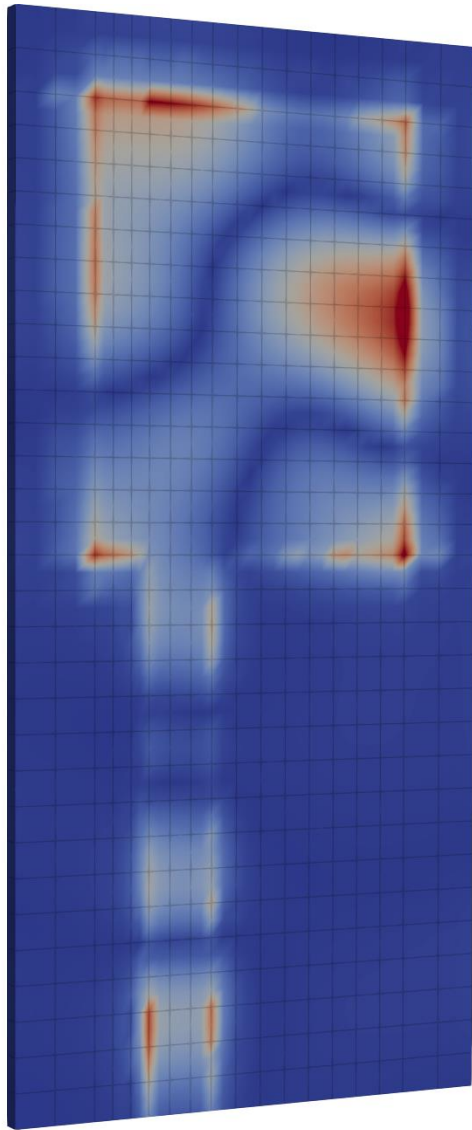


Design of a GPU-Accelerated DGTD Maxwell Solver

Vladimir Yu. Fedorov, Paul Hilscher

Raiden Numerics GK

www.raiden-numerics.com

24.09.2026 • MFEM Community Workshop

Company Profile

Small software company based in **Kyoto, Japan**

Paul Hilscher, PhD (Founder)

LF-EMAG, Plasma, Multiphysics

linkedin.com/in/paul-hilscher

paul.hilscher@raiden-numerics.com



Vladimir Fedorov, PhD (Application Specialist)

HF-EMAG, Photonics

linkedin.com/in/vladimir-yu-fedorov

vladimir.fedorov@raiden-numerics.com

Electromagnetics • Photonics • Plasmas: simulation services, consulting, solver development



Su San Lee / unsplash.com



mufem is a multiphysics solver built on MFEM, with Python as the primary interface for defining simulation cases (materials, boundary conditions, sources) and running them.

- Scriptable end-to-end for: automation, parameter sweeps, and AI-driven design
- Community edition: free for academic and commercial use, covering most workflows.

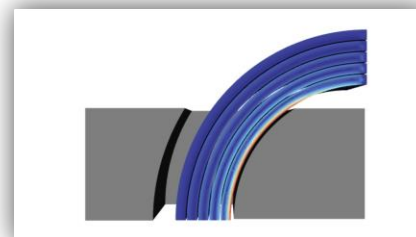


github.com/raiden-numerics/mufem

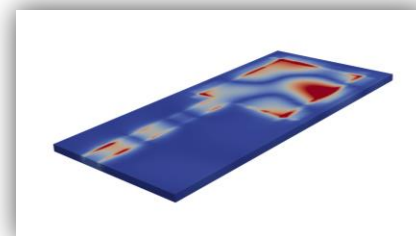
Available Models (Selection)



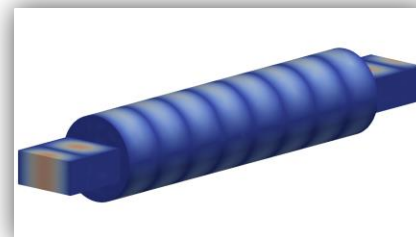
Time-Domain Magnetic
motors, generators, transient currents, coupled electromechanical motion



Time-Harmonic Magnetic
transformers, induction heating, eddy-current losses



Time-Domain Maxwell
broadband signals, photonics, dispersive response — full spectrum in one solve



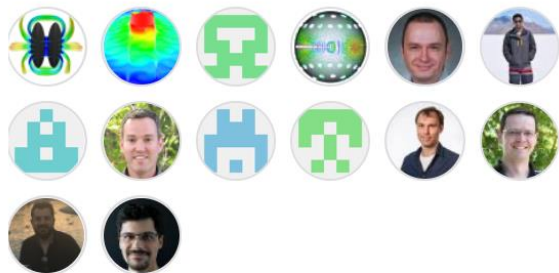
Time-Harmonic Maxwell
antennas, waveguides, and resonators at a single frequency, S-parameters

Why MFEM?

Proven open-source foundation

BSD-licensed, openly developed at LLNL by a large community

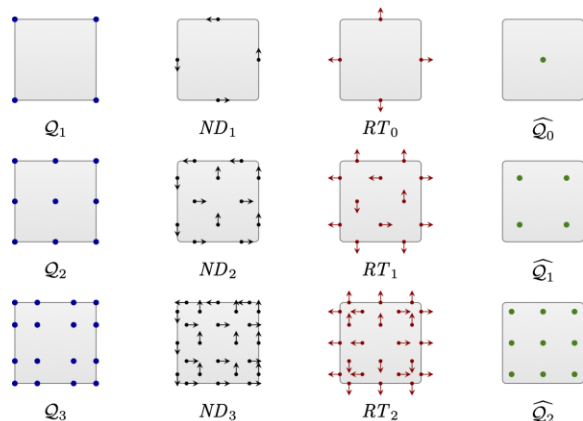
Contributors 191



+ 177 contributors

High-order accuracy

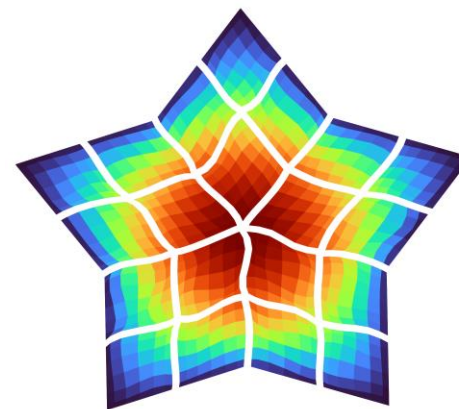
H1, H(curl), H(div) and DG spaces at arbitrary polynomial



<https://mfem.org/tutorial/img/examples1.png>

Parallel and high-performance

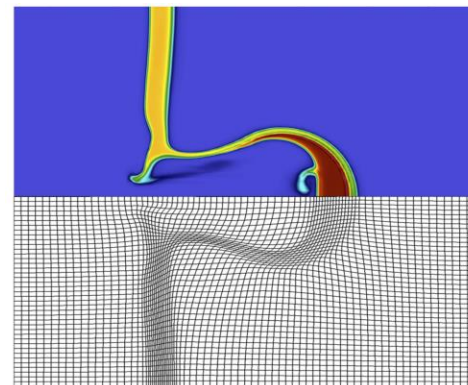
MPI-parallel from laptop to cluster, with GPU-accelerated



<https://glvis.org/img/gallery/quadrature-lor.png>

Adaptive meshes

local refinement in parallel, with load balancing



https://mfem.org/img/gallery/gas_impact.jpg

Solving the transient Maxwell Equations

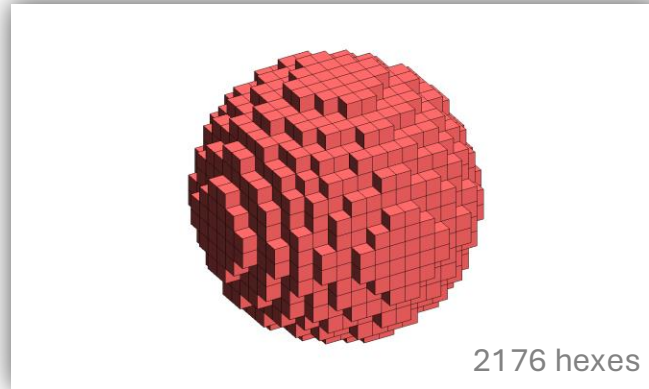
Maxwell equations

$$\mu \frac{\partial \vec{H}}{\partial t} = -\vec{\nabla} \times \vec{E}$$
$$\varepsilon \frac{\partial \vec{E}}{\partial t} = \vec{\nabla} \times \vec{H}$$

6 coupled first order differential equations with tensorial material properties

FDTD

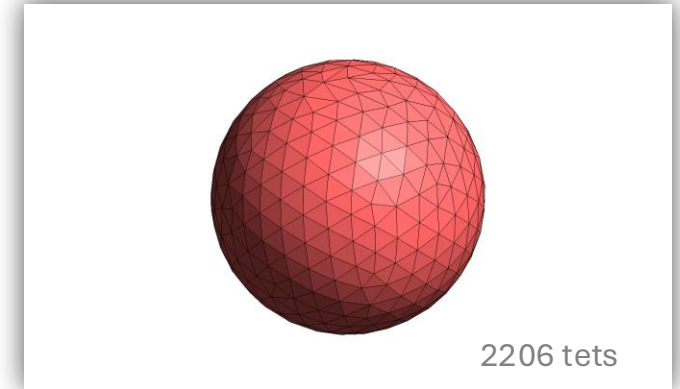
Finite-Difference Time-Domain



- + simple, mature, industry standard
- + simple mesh
- + matrix-free and cheap per cell
- + embarrassingly parallel
- staircases at curved boundaries
- second order only: dispersion builds up
- refinement is global, not local

FETD

Finite-Element Time-Domain



- + curved elements follow the geometry
- + high order: little dispersion over long runs
- + local refinement in h and p
- + allows unstructured mesh
- more costly per degree of freedom
- global mass matrix: a solve every step
- parallel runs need global MPI

DGTD: Discontinuous Galerkin Time-Domain Method

Maxwell equations

$$\begin{aligned}\mu \frac{\partial \vec{H}}{\partial t} &= -\vec{\nabla} \times \vec{E} \\ \varepsilon \frac{\partial \vec{E}}{\partial t} &= \vec{\nabla} \times \vec{H}\end{aligned}$$

Weak form

$$\begin{aligned}\int_{\Omega^e} \mu \frac{\partial \vec{H}}{\partial t} \cdot \vec{\varphi}_i dv + \int_{\Omega^e} \vec{E} \cdot (\vec{\nabla} \times \vec{\varphi}_i) dv + \oint_{\partial\Omega^e} (\hat{n} \times \vec{E}) \cdot \vec{\varphi}_i ds &= 0 \\ \int_{\Omega^e} \varepsilon \frac{\partial \vec{E}}{\partial t} \cdot \vec{\varphi}_i dv - \int_{\Omega^e} \vec{H} \cdot (\vec{\nabla} \times \vec{\varphi}_i) dv - \oint_{\partial\Omega^e} (\hat{n} \times \vec{H}) \cdot \vec{\varphi}_i ds &= 0\end{aligned}$$

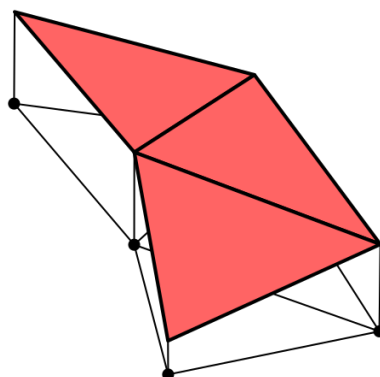
Matrix form

$$\begin{aligned}\mu \mathbf{M} \frac{d\mathbf{H}}{dt} &= -\mathbf{S}\mathbf{E} - \mathbf{F}_H(\mathbf{E}, \mathbf{H}) \\ \varepsilon \mathbf{M} \frac{d\mathbf{E}}{dt} &= \mathbf{S}\mathbf{H} + \mathbf{F}_E(\mathbf{E}, \mathbf{H})\end{aligned}$$

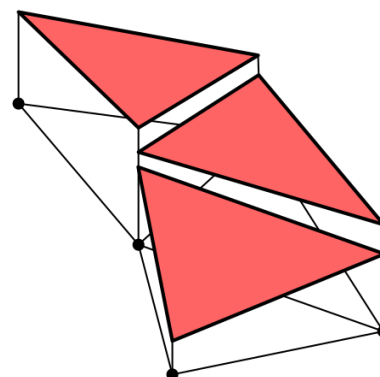
Numerical flux

$$\begin{aligned}\hat{n} \times \vec{E}^* &= \hat{n} \times \frac{Y \vec{E} + Y^+ \vec{E}^+ + \hat{n} \times (\vec{H}^+ - \vec{H})}{Y + Y^+} & Y &= \frac{1}{Z} \\ \hat{n} \times \vec{H}^* &= \hat{n} \times \frac{Z \vec{H} + Z^+ \vec{H}^+ - \hat{n} \times (\vec{E}^+ - \vec{E})}{Z + Z^+} & Z &= \sqrt{\frac{\mu}{\varepsilon}}\end{aligned}$$

CG



DG



- global mass matrix:
a solve at every step
- parallel runs need global
MPI communication

DG parallelizes better on both MPI and GPUs!

- + block-diagonal mass matrix:
no solve
- + element-local:
neighbor communication only
- o explicit time step:
low numerical dispersion
- increased number of DOFs

Survey of MFEM components

Nodal basis

$$\vec{H}(\vec{r}, t) = \sum_i \vec{h}_i(t) l_i(\vec{r})$$

$$\vec{\varphi}_i(\vec{r}) = l_i(\vec{r}) \hat{d}$$

$$\vec{E}(\vec{r}, t) = \sum_i \vec{e}_i(t) l_i(\vec{r})$$

$$d = x, y, z$$

Strong form

$$\int_{\Omega^e} \mu \frac{\partial \vec{H}}{\partial t} \cdot \vec{\varphi}_i dv + \int_{\Omega^e} (\vec{\nabla} \times \vec{E}) \cdot \vec{\varphi}_i dv + \oint_{\partial\Omega^e} (\hat{n} \times (\vec{E}^* - \vec{E})) \cdot \vec{\varphi}_i ds = 0$$

$$\int_{\Omega^e} \varepsilon \frac{\partial \vec{E}}{\partial t} \cdot \vec{\varphi}_i dv - \int_{\Omega^e} (\vec{\nabla} \times \vec{H}) \cdot \vec{\varphi}_i dv - \oint_{\partial\Omega^e} (\hat{n} \times (\vec{H}^* - \vec{H})) \cdot \vec{\varphi}_i ds = 0$$

Flux difference

$$\hat{n} \times (\vec{E}^* - \vec{E}) = \beta_1 \hat{n} \times \llbracket \vec{E} \rrbracket + \beta_2 \hat{n} (\hat{n} \cdot \llbracket \vec{H} \rrbracket) - \beta_2 \llbracket \vec{H} \rrbracket$$

$$\hat{n} \times (\vec{H}^* - \vec{H}) = \gamma_1 \hat{n} \times \llbracket \vec{H} \rrbracket - \gamma_2 \hat{n} (\hat{n} \cdot \llbracket \vec{E} \rrbracket) + \gamma_2 \llbracket \vec{E} \rrbracket$$

Definitions

$$\llbracket \vec{E} \rrbracket = \vec{E}^+ - \vec{E} \quad \beta_1 = \frac{Y^+}{Y + Y^+} \quad \beta_2 = \frac{1}{Y + Y^+}$$

$$\llbracket \vec{H} \rrbracket = \vec{H}^+ - \vec{H} \quad \gamma_1 = \frac{Z^+}{Z + Z^+} \quad \gamma_2 = \frac{1}{Z + Z^+}$$

Bilinear form	Integrator	Availability in MFEM
(u, v)	<code>mfem::MassIntegrator</code>	✓ exists
$\left(\frac{\partial u}{\partial x_d}, v\right)$	<code>mfem::ConvectionIntegrator</code>	✓ exists
$\beta \langle \llbracket u \rrbracket, \llbracket v \rrbracket \rangle$	FN0	✗ custom
$\beta \langle \hat{n}_d \llbracket u \rrbracket, \{v\} \rangle$	FN1	✗ custom
$\beta \langle \hat{n}_{d_0} \hat{n}_{d_1} \llbracket u \rrbracket, \llbracket v \rrbracket \rangle$	FN2	✗ custom

Custom additions to MFEM

$$\text{FN0} \\ \beta \langle \llbracket u \rrbracket, \llbracket v \rrbracket \rangle$$

$$\text{FN1} \\ \beta \langle \hat{n}_d \llbracket u \rrbracket, \{v\} \rangle$$

$$\text{FN2} \\ \beta \langle \hat{n}_{d_0} \hat{n}_{d_1} \llbracket u \rrbracket, \llbracket v \rrbracket \rangle$$

Custom flux face integrators with **partial assembly** support

inherited from
`mfem::BilinearFormIntegrator`

implement
`AssemblePAInteriorFaces`
`AssemblePABoundaryFaces`
`AddMultPA`

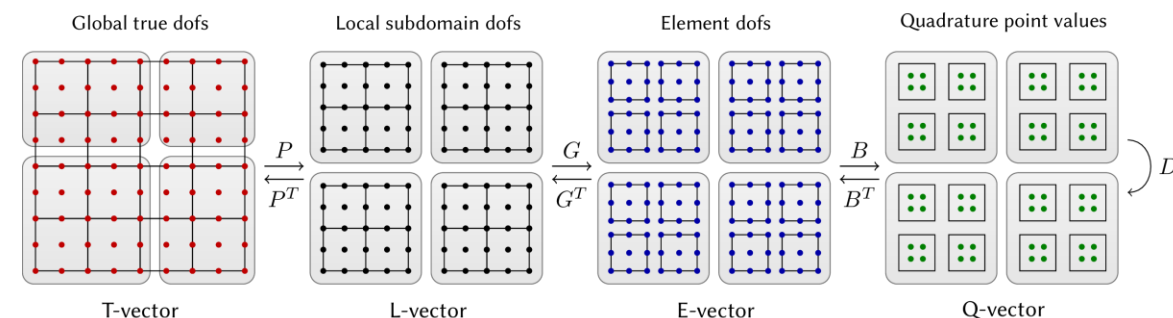
Partial assembly:

- no global matrix
store quadrature-point data, apply the action on the fly
- memory per degree of freedom does not grow with order
smaller memory footprint → less traffic per apply

Additional infrastructure:

- additional `ConvectionIntegrator` PA kernel specializations for curved elements via MFEM's public `AddSpecialization`: works around MFEM #3055
- new `AddInternalBoundaryFaceIntegrator`: a patch to MFEM
embedded zero-thickness sheets: lumped ports, TF/SF sources, interior PEC/PMC

$$A = P^T G^T B^T D B G P$$



<https://mfem.org/performance/>

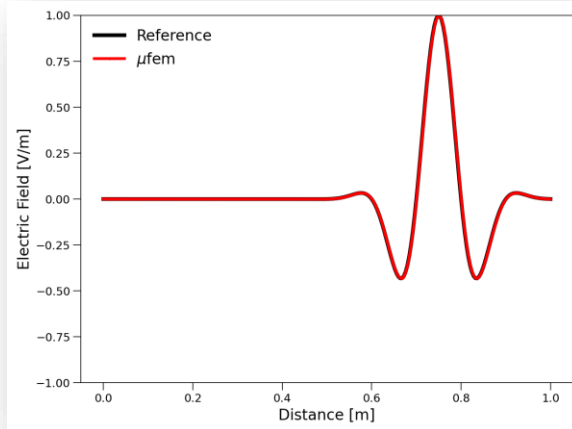
Used as-is:

- no custom time integrator
stock `mfem::ODESolver`
- no device code:
all dispatch from `mfem::forall` family

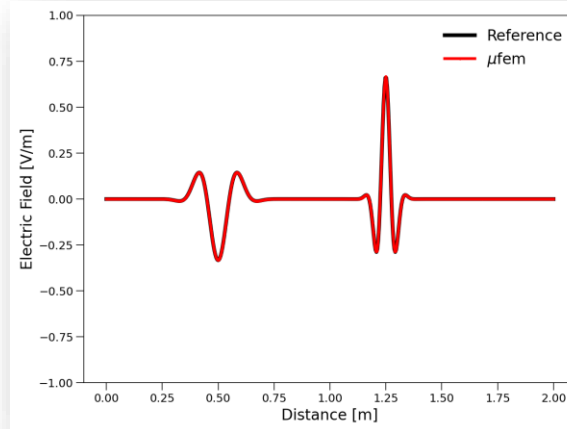
Validation test cases

Validated against analytical solutions and published device data

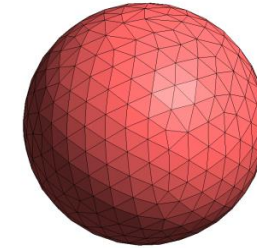
Free-space propagation
numerical dispersion



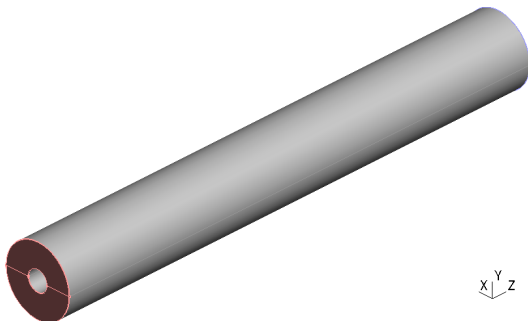
Fresnel reflection
material interfaces



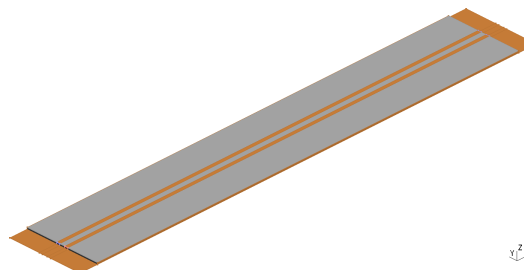
Mie scattering
open spaces, ABC and PML



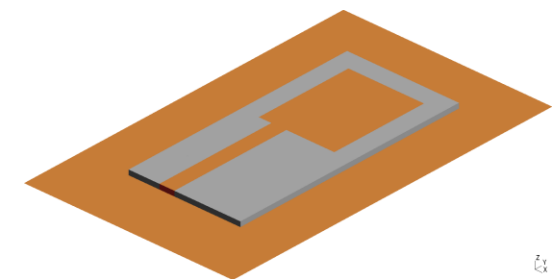
Coaxial cable
port eigenvalue solver



Microstrip line
lumped ports, S-parameters



Patched antenna
far-field radiation sensors



| Shape-changing electronics

Flexible electronics have left the lab:
foldable phones, rollable displays, textile sensors,
medical patches



AI image: Google Gemini

Bendable devices
foldables, rollables, wearables

The shape of devices is no longer fixed:
every component has to keep working while
the device bends



AI image: Google Gemini

Smart textiles
sensors and circuits printed onto fabric

Simulation has to follow the geometry

Conformal microstrip patch antenna

A microstrip patch antenna

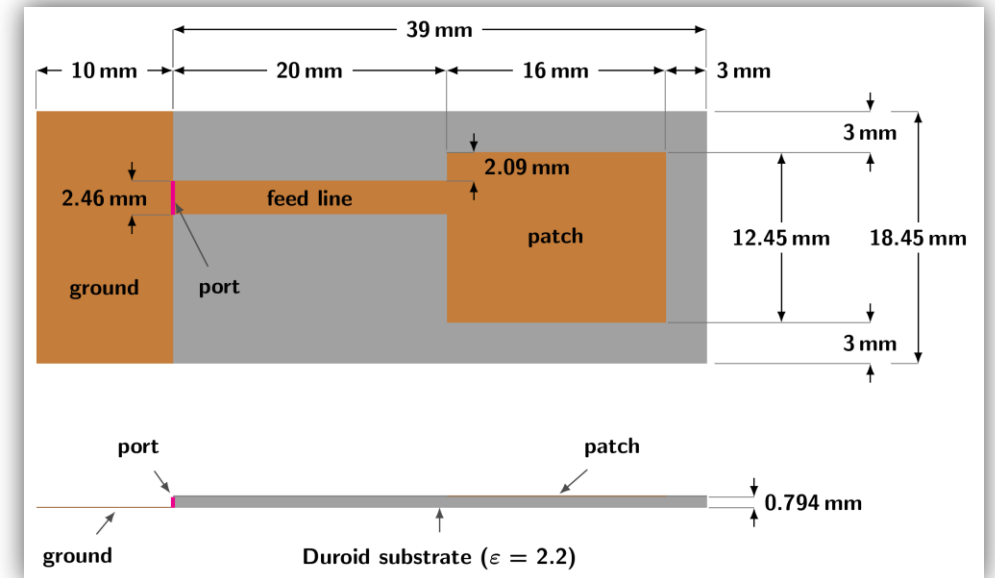
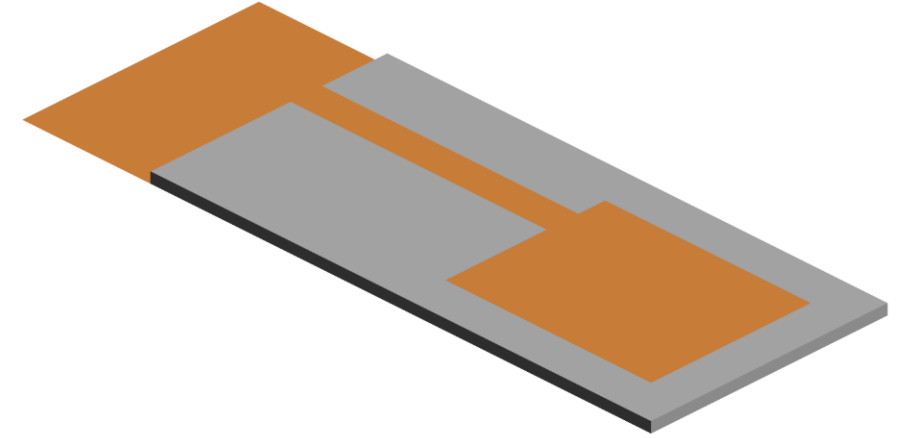
a metal patch on a grounded dielectric substrate, fed by a printed line; the default antenna for compact devices.

The antenna has to bend with the device

a conformal antenna takes the shape of the surface it sits on

What we verify:

- the flat antenna resonates near 7.5 GHz: the measured reference
- the resonant frequency must not move with bend angle: otherwise the device detunes every time it flexes
- the radiation pattern follows the surface: the beam tilts with the bend



D.M. Sheen, IEEE Trans. Microwave Theory Tech. 38, 849 (1990)

Patch antenna simulation setup

The model:

- PML enclosure to model open space
- lumped port feeds the microstrip line
- ground and patch as zero-thickness PEC sheets
- curved hexahedral elements follow the bend

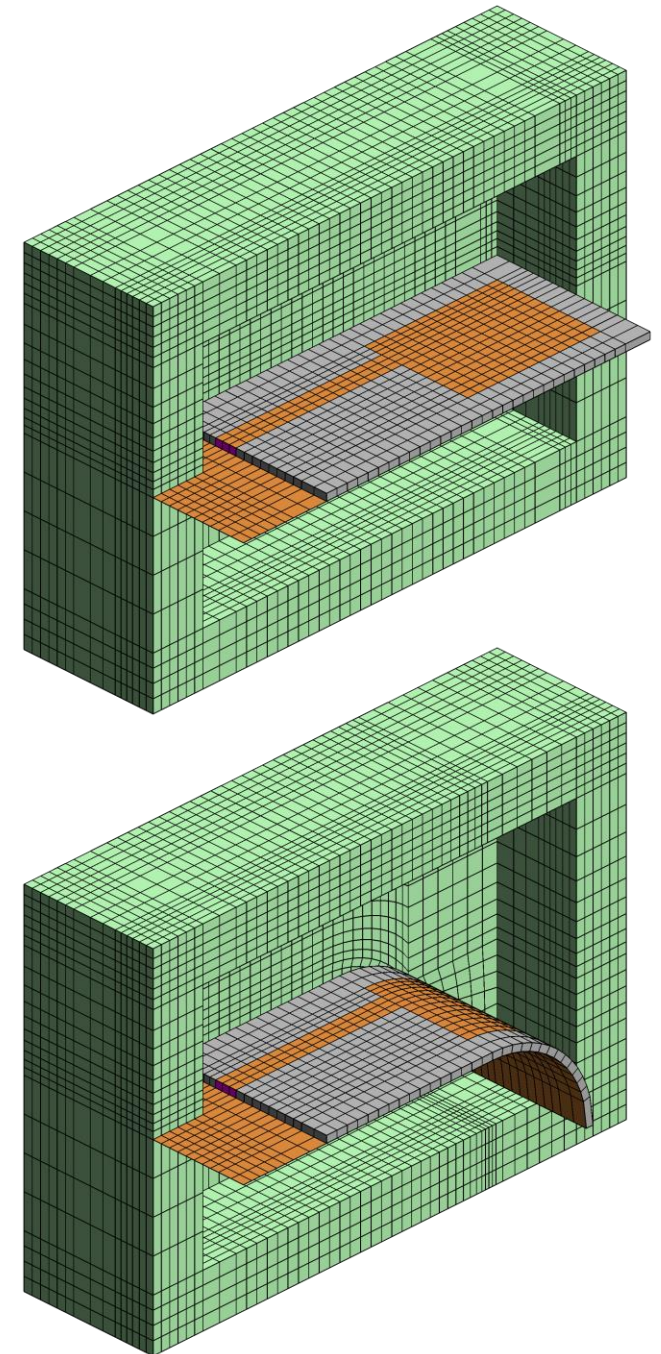
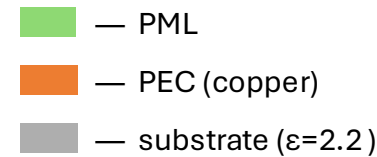
The excitation:

$$V(t) = \exp\left(-\frac{t^2}{\tau^2}\right) \sin(2\pi f t)$$

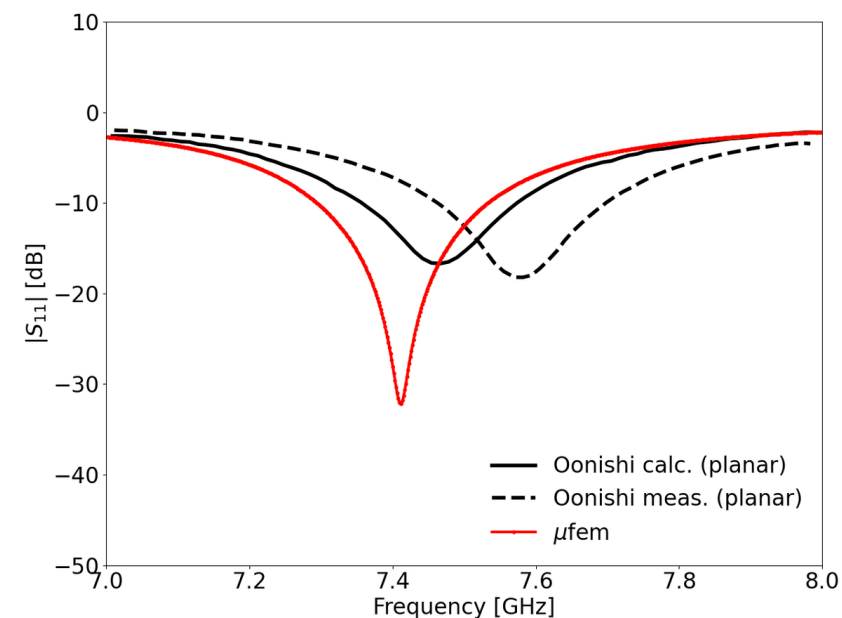
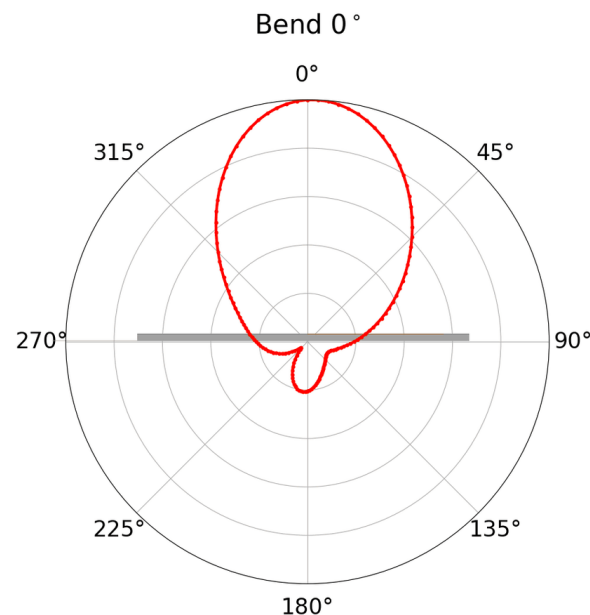
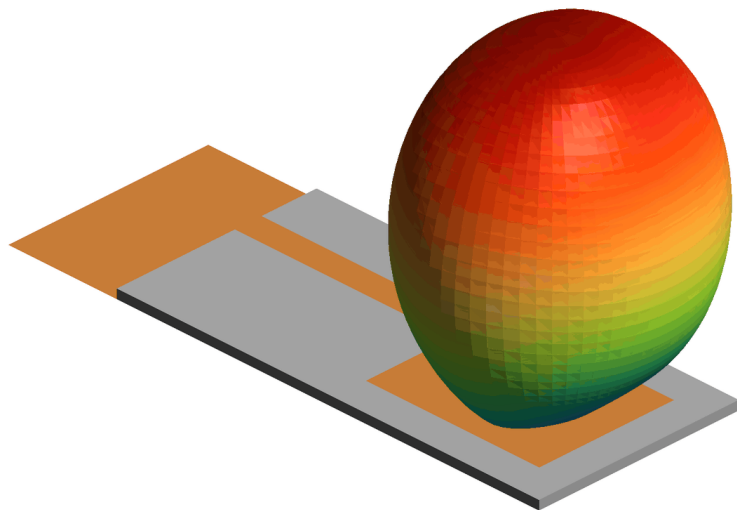
- pulse duration $\tau = 18$ ps
- frequency $f = 8.5$ GHz
- -3 dB bandwidth: 15.5 GHz (6.6 – 22.1 GHz)

Discretization numbers:

- polynomial order 2
- 37,024 hexes $\rightarrow$ 6.0 M degrees of freedom
- 73% PML / 27% physical domain
- time step $\Delta t \propto h/(p+1)^2$



Simulation results

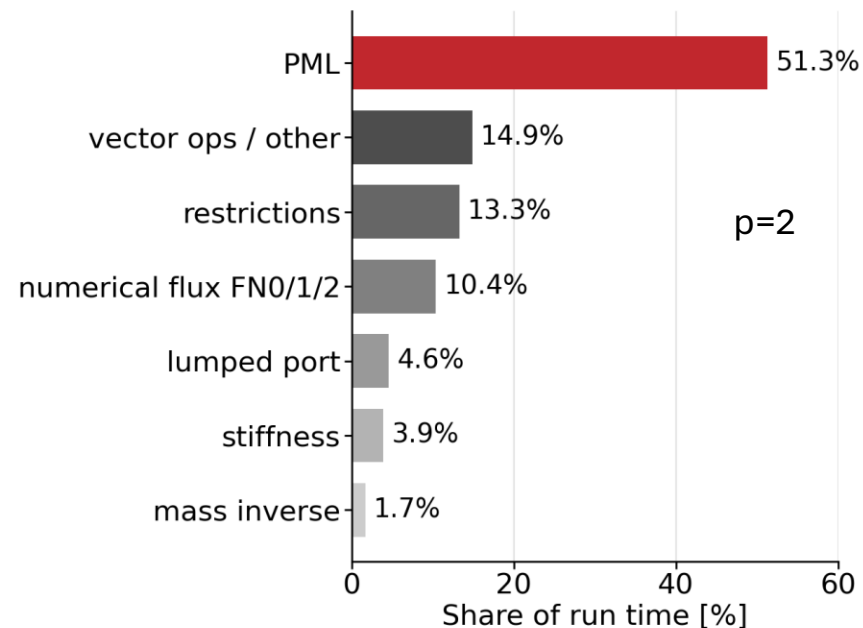
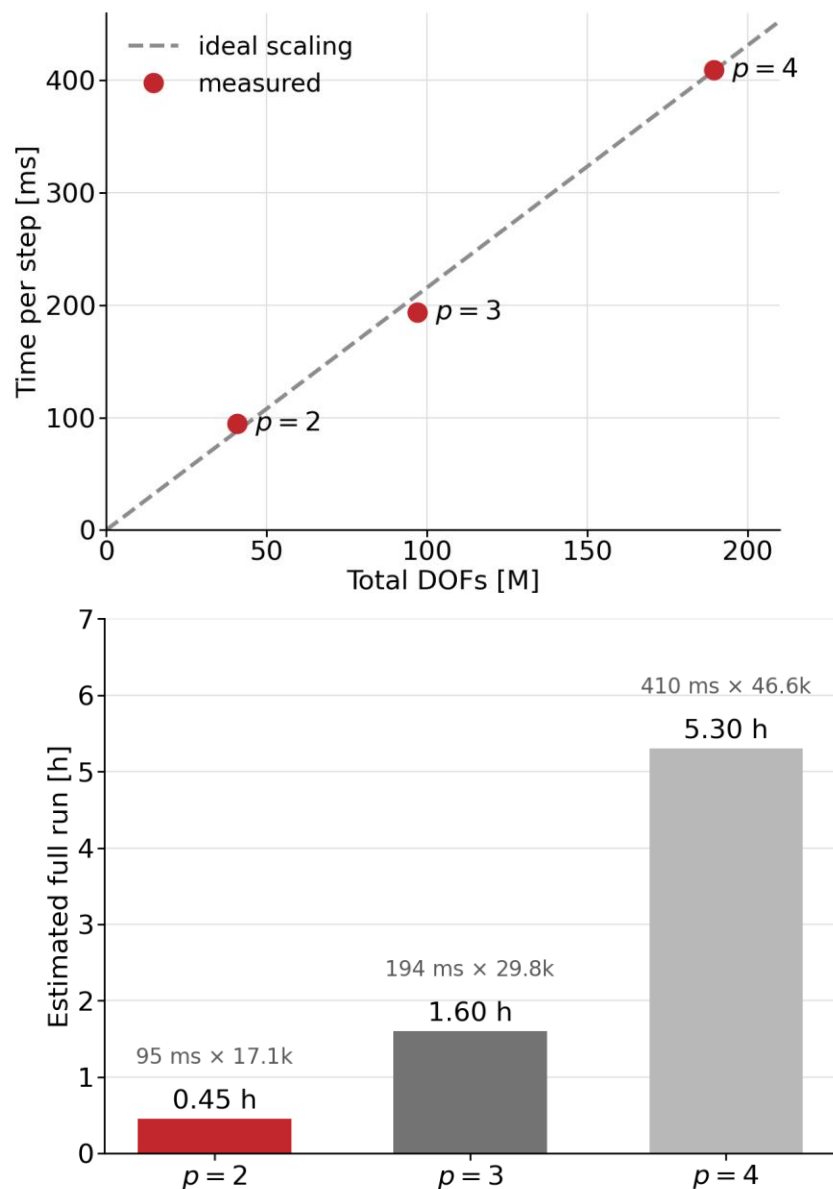


Bend sweep results

- the resonance holds: 7.41 GHz to within 0.2% across the whole 0–90° sweep
- the radiation pattern follows the surface: the main lobe tilts from 0° to ~45° as the board bends through 90°
- the reflection dip stays below -20 dB: no more than 1% reflected

Confirmed: bending steers the antenna without detuning it

Performance overview

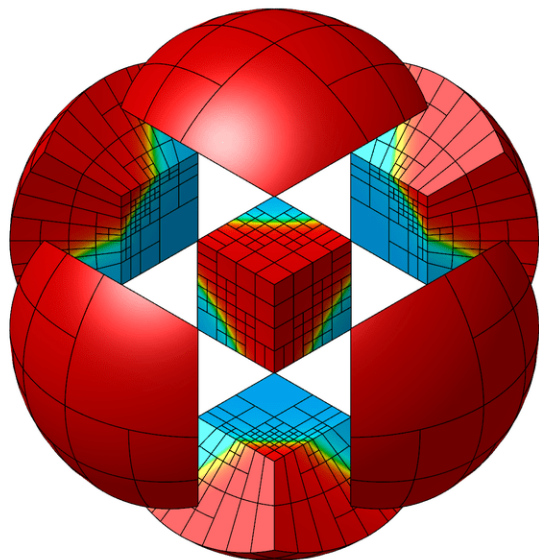


Conclusions

- time per step is linear in degrees of freedom
- PML is half the run: 73 % of the mesh, 51 % of the time
- memory is the limit, not time
 $p = 4$ needs 14.7 GB, $p = 5$ exceeds a 24 GB card

GPU: Nvidia RTX 4090

| Next steps with MFEM



With MFEM

- upstream our patches
- partial assembly on tets



In mufem

- ongoing memory and runtime optimization
- local time stepping
- locally implicit stepping

mufem is built on MFEM. Thank you — and we would like to contribute back.

Links



www.raiden-numerics.com



info@raiden-numerics.com



www.linkedin.com/company/raiden-numerics



github.com/Raiden-Numerics/mufem



pypi.org/project/mufem